PA3 Optimizing MLP using CUDA optimizations

In this assignment, you will implement a simple two-layer MLP in CUDA and optimize it using several techniques.

Before you begin

Familiarize yourself with the CUDA programming framework from the CUDA programming guide and several CUDA tutorials provided online like this basic cuda tutorial. You can practice writing CUDA code on leetgpu.com playground, which runs CUDA code on a GPU simulator. To get performance numbers on a real GPU, you can use a Kaggle notebook with GPU acceleration turned on.

Part A: Forward pass optimizations

In this part, you must write a CUDA program that implements the forward pass of a simple two-layer neural network with a ReLU in between. Given a batch of inputs X (batch size B, each input a vector of size N), your program should compute Y = ReLU(W1 * X), Z = W2 * Y, and return final output Z. Note that the weight matrices W1 and W2 are of size N x N, and the output is also of size N. Your program should consist of separate kernels for the matrix multiplication in the fully connected layers, and for the ReLU computation.

You can initialize the inputs and weights using random numbers. Use a batch size of B = 32, and vary the problem dimension N from 32 to 32K, with multiplicative step size increase by a factor of 2. Measure the total GPU kernel execution time using the CUDA events API (start event before kernel launch, stop and synchronize after kernels complete), and plot this time taken for the batch as a function of N.

Next, optimize your neural network using streams. Split your input into 4 streams of B/4 inputs each, and perform the MLP output computation in each stream in parallel. Once again, compute the total time taken to process the entire batch through the MLP with this streaming implementation, and plot it as a function of N.

Finally, implement your MLP using the PyTorch NN module, and plot the time taken as a function of N. How do the three implementations (batched, streaming, PyTorch) compare with each other? How close to PyTorch's performance are you able to achieve with your CUDA programs? You can try to match the performance of PyTorch with your CUDA code with other optimizations as well, e.g., optimized cuBLAS kernels for matmul, and see how far you can push your MLP program.

Part B: Gradient checkpointing in backward pass

<u>Note:</u> Not included in the syllabus for labquiz2

In this part of the assignment, you will implement the full **forward and backward pass** of a 4-layer MLP in CUDA, then optimize GPU memory usage using **gradient checkpointing**. You will measure and compare memory consumption and runtime between a baseline implementation and a checkpointed one to get an intuition for the *compute vs. memory* trade-off.

The network architecture is:

$$\text{Input} \to \text{FC1} \to \text{ReLU} \to \text{FC2} \to \text{ReLU} \to \text{FC3} \to \text{ReLU} \to \text{FC4} \to \text{Output}$$

with the following dimensions:

Symbol	Value	Meaning
N	1024	Batch size
IN	512	Input features
H_1	2048	Hidden layer 1 width
H_2	2048	Hidden layer 2 width
H_3	2048	Hidden layer 3 width
OUT	512	Output features

Loss: Mean Squared Error (MSE) **Optimizer:** SGD with a learning rate $\eta = 0.001$

Part B.1: Baseline Implementation

In the baseline version, all intermediate activations are allocated with `cudaMalloc` *before* the training loop begins and remain in GPU memory for the entire duration of training. The required buffers are:

Buffer	Shape	Description
Z_1	$N \times H_1$	Pre-activation output of FC1
A_1	$N \times H_1$	Post-ReLU activation after FC1
Z_2	$N \times H_2$	Pre-activation output of FC2
A_2	$N \times H_2$	Post-ReLU activation after FC2
Z_3	$N \times H_3$	Pre-activation output of FC3
A_3	$N \times H_3$	Post-ReLU activation after FC3
Z_4	$N \times \text{OUT}$	Output of FC4 (predictions)

Similarly, preallocate all gradient buffers before the training loop:

```
dW1 (IN x H1),  db1 (H1)
dW2 (H1 x H2),  db2 (H2)
dW3 (H2 x H3),  db3 (H3)
dW4 (H3 x OUT), db4 (OUT)
```

After all allocations are complete, call `cudaMemGetInfo()` to record the amount of GPU memory consumed.

During the forward pass, compute and write into the preallocated buffers in sequence:

```
Z1 = X  @ W1 + b1
A1 = ReLU(Z1)
Z2 = A1 @ W2 + b2
A2 = ReLU(Z2)
Z3 = A2 @ W3 + b3
A3 = ReLU(Z3)
Z4 = A3 @ W4 + b4
```

The backward pass begins by computing the MSE loss and its gradient with respect to the output:

$$L = \frac{1}{|\mathcal{S}|} \sum_i \big(Z_4[i] - y[i]\big)^2, \qquad dZ_4[i] = \frac{2\,\big(Z_4[i] - y[i]\big)}{|\mathcal{S}|}$$

where $|\mathcal{S}| = N \times \mathrm{OUT}$ is the total number of elements. Gradients then propagate through the network as follows. For each layer $\ell = 4, 3, 2, 1$ in reverse order, the pattern is identical:

Layer 4 $(Z_4 = A_3 W_4 + b_4)$:

$$dW_4 = A_3^\top\, dZ_4, \qquad db_4 = \textstyle\sum_{\text{rows}} dZ_4, \qquad dA_3 = dZ_4\, W_4^\top$$

Layer 3 $(Z_3 = A_2 W_3 + b_3,\ A_3 = \mathrm{ReLU}(Z_3))$:

$$dZ_3 = dA_3 \cdot \mathbf{1}[Z_3 > 0], \qquad dW_3 = A_2^\top\, dZ_3, \qquad db_3 = \textstyle\sum_{\text{rows}} dZ_3, \qquad dA_2 = dZ_3\, W_3^\top$$

Layer 2 $(Z_2 = A_1 W_2 + b_2,\ A_2 = \mathrm{ReLU}(Z_2))$:

$$dZ_2 = dA_2 \cdot \mathbf{1}[Z_2 > 0], \qquad dW_2 = A_1^\top\, dZ_2, \qquad db_2 = \textstyle\sum_{\text{rows}} dZ_2, \qquad dA_1 = dZ_2\, W_2^\top$$

Layer 1 $(Z_1 = X W_1 + b_1,\ A_1 = \mathrm{ReLU}(Z_1))$:

$$dZ_1 = dA_1 \cdot \mathbf{1}[Z_1 > 0], \qquad dW_1 = X^\top\, dZ_1, \qquad db_1 = \textstyle\sum_{\text{rows}} dZ_1$$

Finally, apply the SGD update to all parameters:

$$W \leftarrow W - \eta\, dW, \qquad b \leftarrow b - \eta\, db$$

Because all activations $(Z_1, A_1, \ldots, Z_3, A_3)$ remain in GPU memory throughout, the backward pass can read them directly without any recomputation.

Use CUDA events to measure the time spent in the forward pass and the backward pass separately. Run the training loop for at least 100 iterations and find the average per-iteration times.

All weight matrices are stored in row-major order. For a linear layer $Z = A\,W + b$ where A is $M \times K$ and W is $K \times N$, the `matmul` kernel computes $C[i][j] = \sum_k A[i][k] \cdot W[k][j]$, producing a $M \times N$ output.

For the **transposed** multiplications needed in the backward pass, do *not* physically transpose the matrix. Instead, swap the row/column indexing inside the kernel. The two transposed calls you need are:

- $dW_\ell = A_{\ell-1}^\top\, dZ_\ell$: here $A_{\ell-1}$ has shape $N \times K$ and dZ_ℓ has shape $N \times M$, giving dW_ℓ of shape $K \times M$.
- $dA_{\ell-1} = dZ_\ell\, W_\ell^\top$: here dZ_ℓ is $N \times M$ and W_ℓ is $K \times M$, giving $dA_{\ell-1}$ of shape $N \times K$.

Concretely, write two separate kernels (or a single kernel with a `bool transA, transB` flag) rather than trying to reuse the same kernel signature for all four backward multiplications.

Part B.2: Gradient Checkpointing

In gradient checkpointing, rather than storing every intermediate activation, only a small number of *checkpoints* are retained. Activations not at a checkpoint are freed after the forward pass and recomputed on-the-fly during backpropagation.

In the checkpointed version, only two activation buffers are kept in GPU memory at all times:

Buffer	Shape	Reason
A_2	$N \times H_2$	Checkpoint for the second half of the network
Z_4	$N \times \text{OUT}$	Required for loss computation

All other activations (Z_1, A_1, Z_2, Z_3, A_3) are allocated temporarily during computation and freed immediately afterward.

The forward pass function signature should accept all weight/bias pointers, the persistent checkpoint buffers, and the dimension constants. A clean way to organize it:

```
void forward_pass(
    float* X,
    float* W1, float* b1,  // ... W4, b4
    float* A2_checkpoint,  // persistent (kept for backward)
    float* Z4_out,         // persistent (needed for loss)
    int N, int IN, int H1, int H2, int H3, int OUT);
```

Inside the function, `cudaMalloc` temporary buffers for the activations that will be discarded, fill them in sequence, copy the checkpoint value into `A2_checkpoint`, and call `cudaFree` on every temporary at the end.

The backward pass for the checkpointed version must first **recompute** the discarded activations before it can compute any gradients. Structure it in three phases:

Phase 1 — Recompute missing activations:

```
// Allocate temporaries for recomputed values
cudaMalloc(&Z1, N * H1 * sizeof(float));
cudaMalloc(&A1, ...); cudaMalloc(&Z2, ...);
cudaMalloc(&Z3, ...); cudaMalloc(&A3, ...);

// Recompute first half from X
matmul(X,  W1, Z1, N, IN, H1);  add_bias(Z1, b1, N, H1);
memcpy(A1, Z1);  relu(A1, N*H1);
matmul(A1, W2, Z2, N, H1, H2);  add_bias(Z2, b2, N, H2);
// Note: A2 is already available as the checkpoint

// Recompute Z3 and A3 from the checkpoint A2
matmul(A2_ckpt, W3, Z3, N, H2, H3);  add_bias(Z3, b3, N, H3);
memcpy(A3, Z3);  relu(A3, N*H3);
```

Phase 2 — Backpropagate through all four layers (identical to baseline once activations are available).

Phase 3 — Free all temporaries and apply SGD updates.

Call `cudaMemGetInfo()` *after* all persistent buffers have been allocated (weights, biases, checkpoints, gradient buffers) but *before* the training loop starts.

Compare baseline vs checkpointed across forward time, backward time, total time, and GPU memory used. How much memory does checkpointing save, and what is the computational overhead

on the backward pass? How does the measured overhead compare to what you would theoretically expect, given the amount of recomputation performed? Finally, think through the trade-off: when would gradient checkpointing be worth using, and when would it not be?

Note: CUDA kernels execute asynchronously. Always use `cudaEventSynchronize` on the stop event before reading the elapsed time, and warm up the GPU with one un-timed iteration before starting the measurement loop.